

→ [Session 04 페이지 바로가기]

Session 4 – Application Deployment

1. 세션 목적

- Rancher UI를 통해 Project와 Namespace를 생성합니다.
- Rancher Dashboard의 Workloads 개념을 이해합니다.
- Rancher의 Continuous Delivery를 통해 Helm chart 기반 애플리케이션을 배포합니다.
- 배포된 애플리케이션의 상태 확인 및 접근 방법을 익힙니다.



이 세션에서 Rancher가 Kubernetes 위에 추가하는 것

Kubernetes를 이미 사용하고 있다면 `kubectl`, Helm CLI, YAML 파일 작업에 익숙할 것입니다. SUSE Rancher Prime은 이 워크플로우를 대체하는 것이 아니라, 동일한 작업을 **멀티 클러스터 환경에서 GUI로 일관되게 수행**할 수 있도록 추상화 계층을 제공합니다. 이 세션에서는 Rancher가 순수 Kubernetes 위에 어떤 개념과 UI를 추가하는지 직접 확인합니다.

2. 실습 전 확인 사항

- Session 3 완료 (Downstream Cluster(`custom-test`)가 **Active** 상태)
- `sr-gui-xx` VM VNC Console 에서 Rancher Dashboard 접속 가능
- Downstream Cluster(`custom-test`)의 노드 **Active** 상태

항목	확인
<code>sr-gui-xx</code> VM VNC Console에서 Rancher Dashboard 접속 가능	☐
Downstream Cluster(<code>custom-test</code>) 상태: Active	☐
노드 Active 상태	☐

3. Hands-on 시나리오



Session 04 실습은 이전 세션과 동일하게 `sr-gui-XX` VM 에서 진행합니다.

VM Password: `SRedu!!!`

Dashboard admin Password: `RancherEdu2026!`

Step 1. Project 생성하기

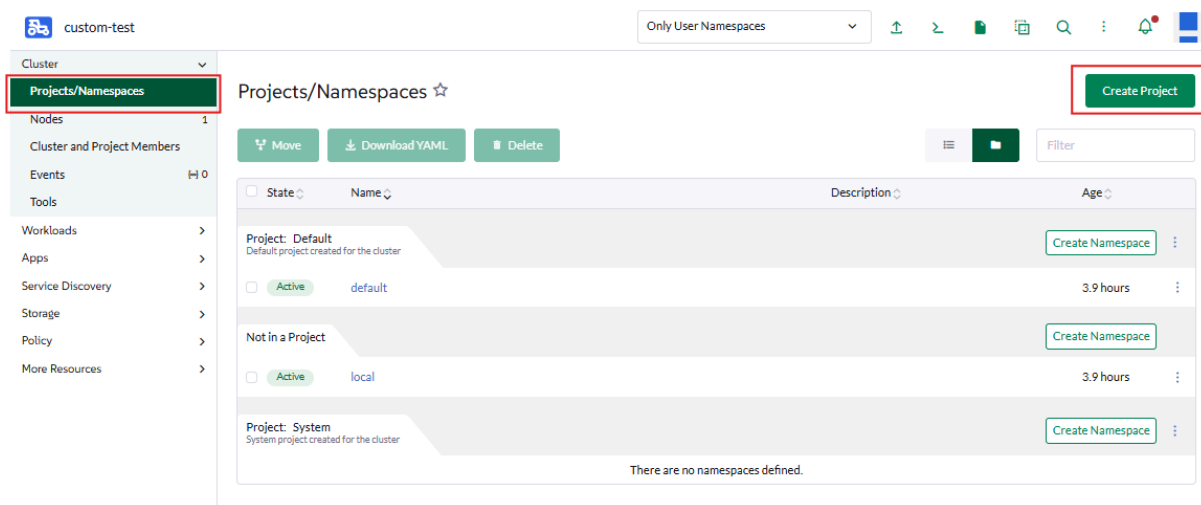
🔍 Rancher의 Project란?

순수 Kubernetes에는 `Namespace` 단위만 존재합니다. Rancher는 여러 Namespace를 하나의 논리 단위로 묶는 **Project**라는 개념을 추가합니다. Project를 사용하면 팀 단위로 RBAC 정책과 리소스 쿼터를 한 번에 적용할 수 있습니다. 클러스터 생성 시 `Default`와 `System` 두 개의 Project가 자동 생성됩니다. `System` Project는 Rancher 내부 컴포넌트가 사용하므로 수정하지 않습니다.

Rancher Main Dashboard에서 클러스터 목록에서 `downstream-cluster` 를 클릭합니다.

좌측 메뉴에서 **Cluster** → **Projects/Namespace** 를 클릭합니다.

화면 우측 상단의 **Create Project** 버튼을 클릭합니다.



Project 설정 화면에서 다음을 입력합니다.

- **Name:** `hands-on-project`

Project: Create

Name*

hands-on-project

Description

Any text you want that better describes this resource

Members

Resource Quotas

Container Default Resource Limit

Labels & Annotations

Members

User	Role
Default Admin (admin)	Project Owner
Local	

Add

Cancel

Create

Create 버튼을 클릭하여 Project를 생성합니다.

✓ **체크포인트:** `hands-on-project` 가 Project 목록에 표시되는지 확인합니다.

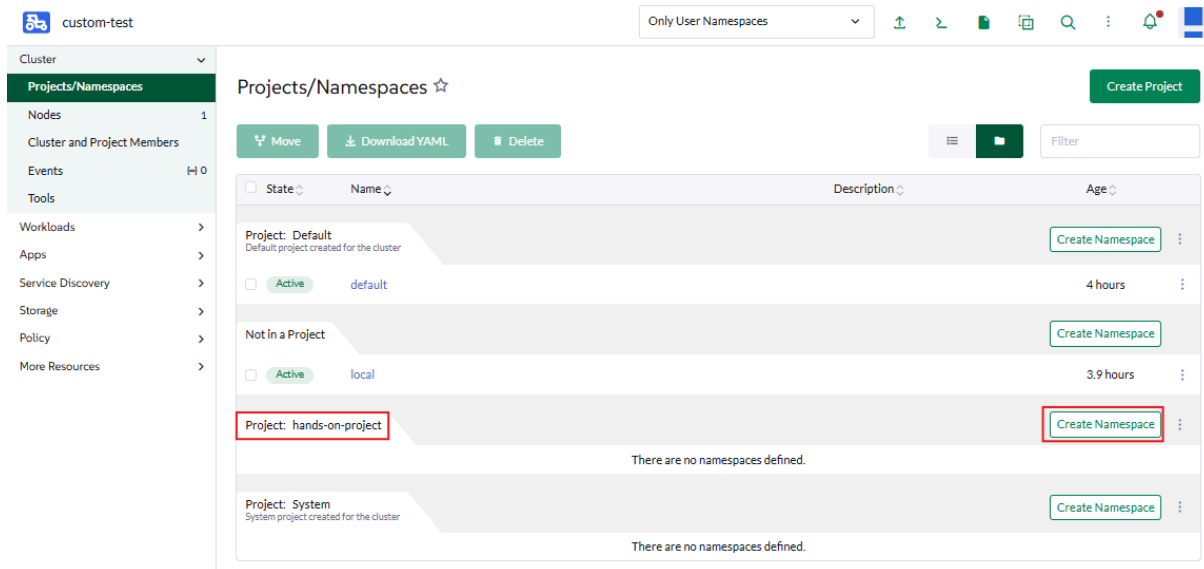
Step 2. Namespace 생성하기

🔍 Rancher에서 Namespace를 만드는 방법이 다른 이유

순수 Kubernetes에서는 `kubectl create namespace <name>` 명령으로 Namespace를 생성하면 어떤 Project에도 속하지 않습니다. Rancher에서는 **Projects/Namespace** 화면에서 특정 **Project** 행의 **Create Namespace** 버튼을 사용해야 해당 Project 소속으로 Namespace가 생성됩니다. Project 소속이 설정되어야 RBAC 및 리소스 쿼터가 Namespace에 상속됩니다.

Projects/Namespace 목록에서 방금 생성한 `hands-on-project` 행을 찾습니다.

`hands-on-project` 행 우측의 **Create Namespace** 버튼을 클릭합니다.



Namespace 설정 화면에서 다음을 입력합니다.

- **Name:** demo-app

Create 버튼을 클릭합니다.

Namespace: Create

✓ **체크포인트:** demo-app Namespace가 hands-on-project 하위에 표시되는지 확인합니다.

Step 3. Workloads 메뉴에서 샘플 애플리케이션 배포하기

Rancher Dashboard의 Workloads 메뉴를 사용하여 nginx 기반 샘플 애플리케이션을 GUI로 배포합니다.

Rancher Workloads 메뉴란?

순수 Kubernetes에서는 Deployment YAML을 작성하거나 `kubectl create deployment` 명령을 사용합니다. Rancher의 **Workloads** 메뉴는 동일한 Deployment, DaemonSet, StatefulSet 등을 **폼(Form) 기반 UI**로 생성할 수 있게 합니다. 내부적으로는 동일한 Kubernetes 오브젝트가 생성되며, Rancher UI에서 만든 리소스를 `kubectl`로 조회하면 동일하게 확인됩니다.

Workloads	용도 및 특징	비유/예시
Pod (파드)	쿠버네티스의 가장 작은 배포 단위로, 하나 이상의 컨테이너를 실행합니다.	앱이 실행되는 가장 작은 단위
Deployment	상태가 없는(Stateless) 앱을 배포하며, 복제본 개수 관리와 업데이트에 최적화되어 있습니다.	일반적인 웹 서버 (Nginx 등)
StatefulSet	상태 유지가 필요한(Stateful) 앱을 배포하며, 고정된 이름과 전용 저장소를 가집니다.	데이터베이스 (MySQL, Redis 등)
DaemonSet	클러스터 내의 모든(또는 특정) 노드 에 파드를 하나씩 반드시 실행합니다.	로그 수집기, 모니터링 에이전트
Job	한 번 실행 하고 작업이 완료되면 종료되는 일회성 작업을 수행합니다.	데이터 백업, 마이그레이션
CronJob	정해진 시간에 맞춰 주기적으로 Job을 실행하는 예약 작업입니다.	매일 새벽 정기 점검, 알림 발송

Step 3-1. Deployment 생성하기

화면 좌측 메뉴에서 **Workloads** → **Deployments** 를 클릭합니다.

화면 우측 상단의 **Create** 버튼을 클릭합니다.

The screenshot shows the Rancher UI interface. On the left, a sidebar contains a navigation menu with items like Cluster, Workloads, CronJobs, DaemonSets, Deployments (selected), Jobs, StatefulSets, and Pods. The main content area is titled 'Deployments' and features a 'Create' button. Below the title are buttons for 'Redeploy', 'Download YAML', and 'Delete'. A table lists several active deployments, including 'capi-controller-manager', 'cert-manager', 'cert-manager-cainjector', 'cert-manager-webhook', 'fleet-agent', 'fleet-controller', 'gitjob', and 'helmops'. Each row shows the deployment's state, name, namespace, image, and readiness metrics.

State	Name	Namespace	Image	Ready	Up To Date	Available	Age
Active	capi-controller-manager	cattle-capi-system	registry.rancher.com/rancher/cluster-api-controller:v1.10.6	1/1	1	1	27 days
Active	cert-manager	cert-manager	quay.io/jetstack/cert-manager-controller:v1.19.1	1/1	1	1	27 days
Active	cert-manager-cainjector	cert-manager	quay.io/jetstack/cert-manager-cainjector:v1.19.1	1/1	1	1	27 days
Active	cert-manager-webhook	cert-manager	quay.io/jetstack/cert-manager-webhook:v1.19.1	1/1	1	1	27 days
Active	fleet-agent	cattle-fleet-local-system	registry.rancher.com/rancher/fleet-agent:v0.14.3	1/1	1	1	27 days
Active	fleet-controller	cattle-fleet-system	registry.rancher.com/rancher/fleet:v0.14.3	1/1	1	1	27 days
Active	gitjob	cattle-fleet-system	registry.rancher.com/rancher/fleet:v0.14.3	1/1	1	1	27 days
Active	helmops	cattle-fleet-system	registry.rancher.com/rancher/fleet:v0.14.3	1/1	1	1	27 days

Deployment 생성 화면이 열리면 상단 탭 중 **General** 탭에서 다음을 입력합니다.

- **Name:** `nginx-demo`
- **Replicas:** `2`
- **Namespace:** `demo-app`

Container Image 섹션에서 다음을 입력합니다.

- **Container Name:** `nginx-demo`
- **Image:** `nginx:latest`

💡 팁: 생성 화면 상단의 탭(Ports, Environment Variables, Health Check, Volumes 등)은 Kubernetes Deployment spec의 각 필드에 대응합니다. YAML 을 직접 작성할 때와 동일한 내용을 폼으로 채우는 방식입니다. 우측 상단의 **Edit as YAML** 버튼을 클릭하면 폼 입력 내용이 YAML로 즉시 변환되어 표시됩니다.

Deployment: Create

Namespace *
demo-app

Name *
nginx-demo

Description
Any text you want that better describes this resource

Replicas *
2

Deployment

Pod

nginx-demo

+ Add Container

General

Health Check

Resources

Security Context

Storage

General

Container Name
nginx-demo

Image
Container Image
nginx:latest

Pull Policy
Always

Init Container

Standard Container

Pull Secrets

Cancel

Edit as YAML

Create

Pod 메뉴에서 **Add Label**을 선택합니다.

Deployment: Create

Namespace *
demo-app

Name *
nginx-demo

Description
Any text you want that better describes this resource

Replicas *
2

Deployment

Pod

nginx-demo

+ Add Container

Labels & Annotations

Networking

Node Scheduling

Pod Scheduling

Resources

Scaling and Upgrade Policy

Security Context

Storage

Labels & Annotations

Pod Labels
Add Label

Pod Annotations
Add Annotation

아래와 같이 **Label**을 작성합니다.

- **Key:** app
- **Value:** nginx-demo

Deployment: Create

Namespace *
demo-app

Name *
nginx-demo

Description
Any text you want that better describes this resource

Replicas *
2

Deployment

Pod

nginx-demo

+ Add Container

Labels & Annotations

Networking

Node Scheduling

Pod Scheduling

Resources

Scaling and Upgrade Policy

Security Context

Storage

Labels & Annotations

Pod Labels

Key

Value

app

nginx-demo

Remove

Add Label

Pod Annotations

Add Annotation

Deployment 메뉴에서 **Add Label**을 선택합니다.

Deployment: Create

Namespace *
demo-app

Name *
nginx-demo

Description
Any text you want that better describes this resource

Replicas *
2

Deployment

Pod

nginx-demo

+ Add Container

Labels & Annotations

Scaling and Upgrade Policy

Labels & Annotations

Labels

Key/value pairs that are attached to objects which specify identifying attributes.

Add Label

Annotations

Add Annotation

아래와 같이 **Label**을 작성하고 **Create** 버튼을 클릭하여 Deployment를 생성합니다.

- **Key:** app
- **Value:** nginx-demo

Deployment: Create

Namespace *
demo-app

Name *
nginx-demo

Description
Any text you want that better describes this resource

Replicas *
2

Deployment

Pod

nginx-demo

+ Add Container

Labels & Annotations

Scaling and Upgrade Policy

Labels & Annotations

Labels

Key/value pairs that are attached to objects which specify identifying attributes.

Key

Value

app

nginx-demo

Remove

Add Label

Annotations

Add Annotation

Cancel

Edit as YAML

Create

Deployments 목록으로 돌아와 **nginx-demo** 의 **Ready** 컬럼이 **2/2** 로 표시될 때까지 기다립니다.

demo-appX

Deployments ☆

Create

Redeploy

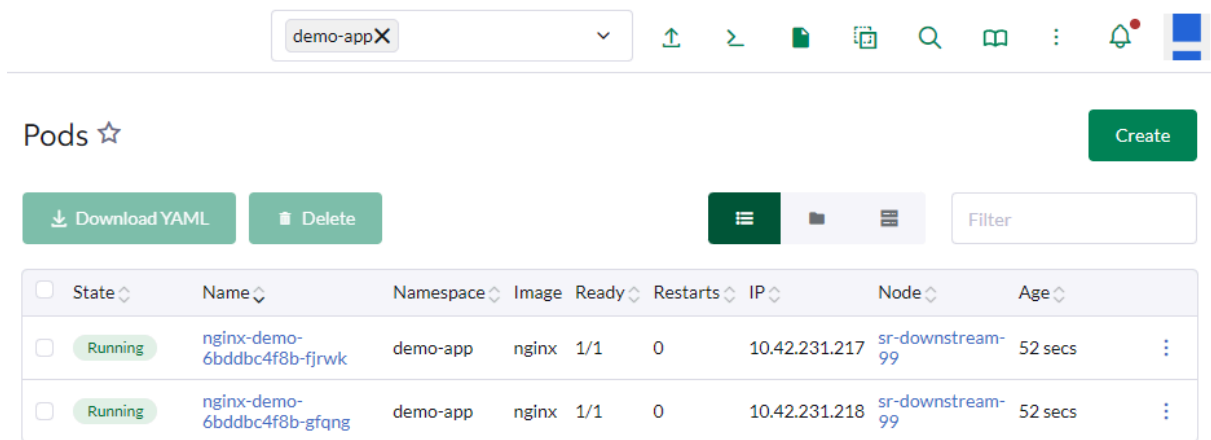
Download YAML

Delete

Filter

☐	State	Name	Namespace	Image	Ready	Up To Date	Available	Age	
☐	Active	nginx-demo	demo-app	nginx	2/2	2	2	19 secs	

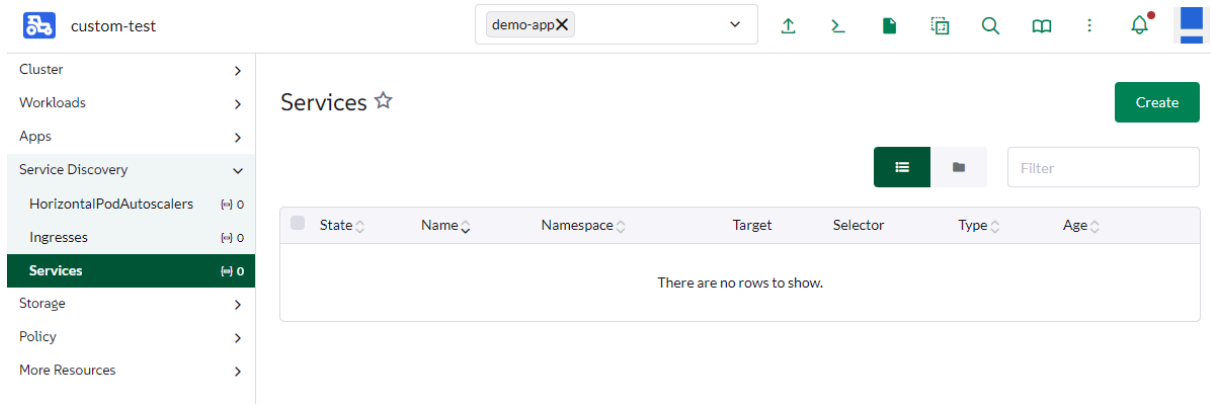
좌측 메뉴에서 **Workloads** → **Pods** 를 클릭합니다. Namespace 필터를 **demo-app** 으로 설정하고, Pod 2개가 모두 **Running** 상태인 것을 확인합니다.



Step 3-2. Service 생성하기

Rancher의 Service Discovery 메뉴란?

Service Type	주요 특징 및 접근 범위
ClusterIP	클러스터 내부 통신 전용 (기본값)
NodePort	노드의 특정 포트를 개방하여 외부 접속 허용
LoadBalancer	외부 로드밸런서를 통해 할당된 대표 주소로 접속 허용
Headless	주로 StatefulSet에 사용하며 단일 통합 IP(ClusterIP)를 생성하지 않고, 연결된 Pod들의 개별 IP를 직접 반환
ExternalName	클러스터 외부의 도메인 주소(DNS)를 내부 서비스처럼 별칭으로 연결하여 사용



Service 유형 선택 화면에서 **ClusterIP** 를 클릭합니다.

Service: Create

Services allow you to define a logical set of Pods that can be accessed with a single IP address and port.

IP	Cluster IP Expose a set of Pods to other Pods within the cluster. This type of Service is only reachable from within the cluster. This is the default type.	EN	External Name Create a Service that uses a DNS name instead of selectors. This is an advanced use case.
H	Headless Create a Service without a cluster IP or load balancer. This is an advanced use case.	LB	Load Balancer Create a load balancer in the underlying infrastructure (e.g. a cloud provider's load balancer) and assign a public IP address to the service. Allows external clients to access the service using the public IP address and the port specified in the service definition.
NP	Node Port Expose the service on each node's IP at a static port.		

Cancel

Service 생성 화면의 **General** 탭에서 다음을 입력합니다.

- **Name:** nginx-demo
- **Namespace:** demo-app

Listener 섹션에서 다음을 입력합니다.

- **Port Name:** nginx-port
- **Listening Port:** 80
- **Target Port:** 80

Service: Create ClusterIP

Services allow you to define a logical set of Pods that can be accessed with a single IP address and port.

Namespace *
demo-app

Name *
helm-demo

Description
Any text you want that better describes this resource

Service Ports

IP Addresses

Selectors

Session Affinity

Labels & Annotations

Service Ports

Port Name
nginx-port

Listening Port
80

Protocol
TCP

Target Port
80

Add

Remove

Selectors 섹션에서 Deployment 생성 시 입력한 Label 값을 입력하여 앞서 생성한 Deployment와 연결 후 **Create** 버튼을 클릭합니다.

- **Key:** app
- **Value:** nginx-demo

Service: Create ClusterIP

Services allow you to define a logical set of Pods that can be accessed with a single IP address and port.

Namespace *
demo-app

Name *
helm-demo

Description
Any text you want that better describes this resource

Service Ports

IP Addresses

Selectors

Session Affinity

Labels & Annotations

Selectors

Selector keys and values are intended to match labels and values on existing Pods.

Matches 2 of 3 existing pods, including "nginx-demo-57fc46c8dc-dq2sq"

Key
app

Value
nginx-demo

Add

Read from File

Remove

Cancel

Edit as YAML

Create

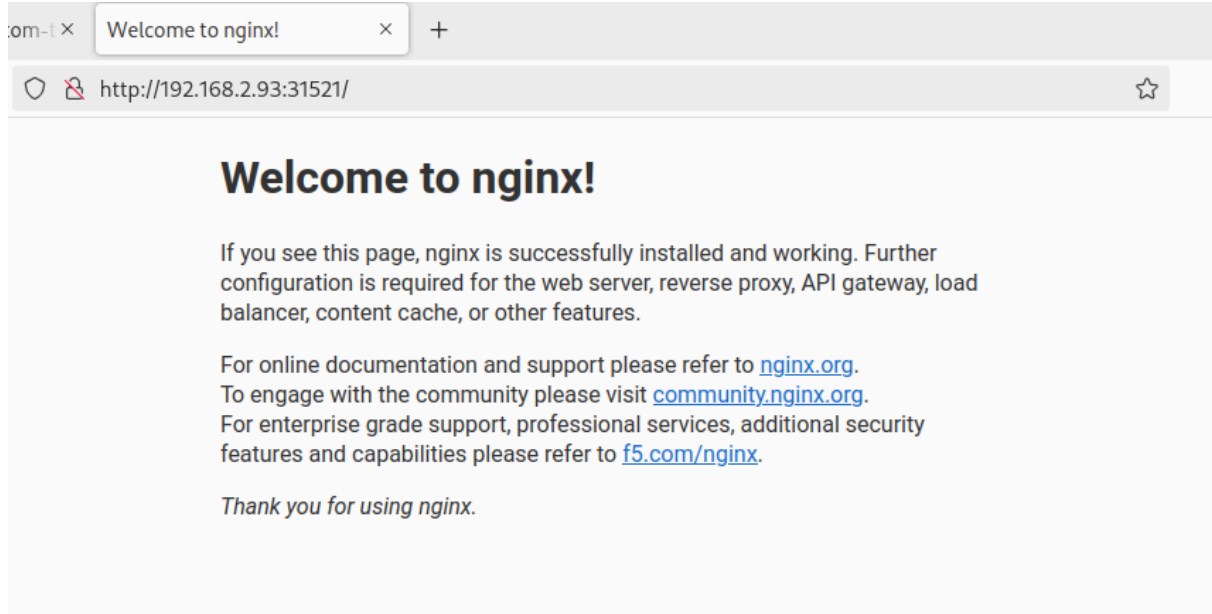
Services 목록에서 생성된 `nginx-demo` Service를 확인하여 **Target** 컬럼에 표시된 nginx-port를 클릭합니다.

Download YAML

Delete

Filter

State	Name	Namespace	Target	Selector	Type	Age
Active	helm-demo	demo-app	nginx-port 80/TCP	app=nginx-demo	Cluster IP	19 secs
Active	nginx-helm	demo-app	32730/TCP 32364/TCP	app.kubernetes.io/instance=nginx-helm app.kubernetes.io/name=nginx	Node Port	23 hours

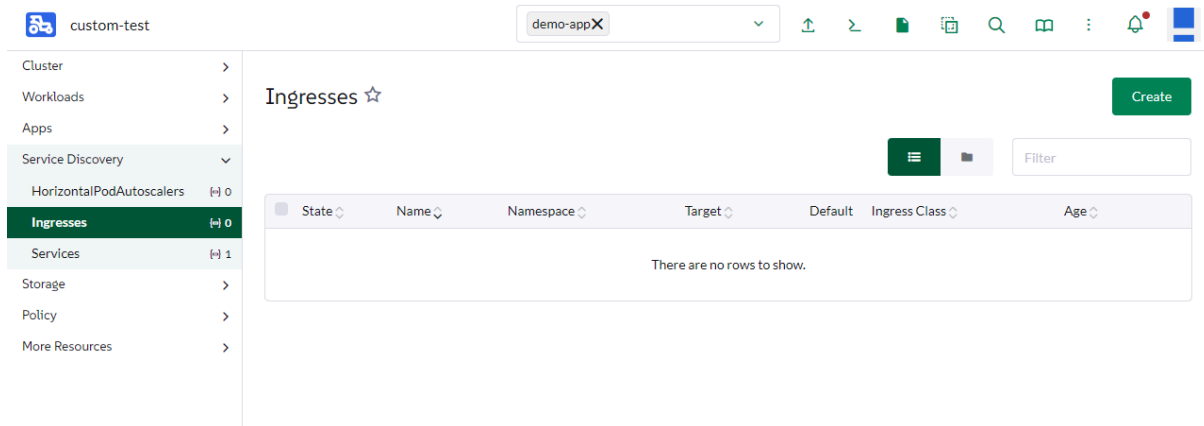


✓ **체크포인트 1:** 브라우저에서 nginx 기본 페이지("Welcome to nginx!")가 표시되는지 확인합니다.

Step 4. Ingress 생성 및 접속 테스트

Step 4-1. Ingress 메뉴 진입

Service Discovery > Ingresses 메뉴로 이동하여 우측 상단의 **Create** 버튼을 클릭합니다.



Step 4-2. 기본 정보 및 라우팅 규칙(Rule) 설정

Ingress를 생성하기 위한 기본 정보와 트래픽을 전달할 서비스 규칙을 설정합니다.

- **Namespace:** `demo-app` 선택
- **Name:** `nginx-ingress` 입력
- **Rules 설정:** * **Request Host:** 접속에 사용할 도메인인 `nginx.test.local` 입력
 - **Path:** `/` (기본값 유지)
 - **Target Service:** 미리 생성해둔 `nginx-demo` 서비스 선택
 - **Port:** `80` 선택

Ingress: Create

Ingresses route incoming traffic from the Internet to Services within the cluster based on the hostname and path specified in the request. You can expose multiple Services on the same external IP address and port.

Namespace *
demo-app

Name *
nginx-ingress

Description
Any text you want that better describes this resource

Rules

Default Backend

Certificates

Ingress Class

Labels & Annotations

Rules

Request Host
nginx.test.local

Path
Prefix /

Add Path

Target Service
nginx-demo

Port
80

Add Rule

Step 4-3. Ingress Class 설정

좌측 메뉴에서 **Ingress Class** 탭으로 이동한 뒤, 드롭다운 메뉴에서 `nginx` 를 반드시 선택합니다. 설정이 완료되면 우측 하단의 **Create** 버튼을 클릭합니다.

Ingress: Create

Ingresses route incoming traffic from the internet to Services within the cluster based on the hostname and path specified in the request. You can expose multiple Services on the same external IP address and port.

Namespace *
demo-app

Name *
nginx-ingress

Description
Any text you want that better describes this resource

Rules

Default Backend

Certificates

Ingress Class

Labels & Annotations

Ingress Class

nginx

Cancel

Edit as YAML

Create

Step 4-4. 생성 상태 확인

Ingress 목록 화면으로 돌아와 방금 생성한 `nginx-ingress` 의 상태가 **Active**인지, Target 주소가 올바르게 연결되었는지 확인합니다.

Ingresses ☆ Create

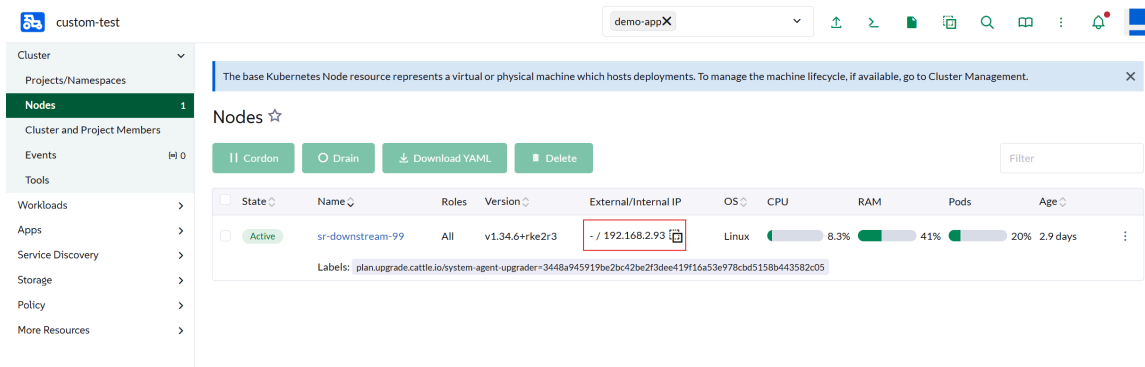
Download YAML Delete Filter

☐	State	Name	Namespace	Target	Default	Ingress Class	Age	
☐	Active	nginx-ingress	demo-app	http://nginx.test.local/ > nginx-demo	—	nginx	4.9 mins	⋮

Step 4-5. DNS 로컬 매핑 (hosts 파일 수정)

실제 도메인 서버가 없는 실습 환경이므로, 설정한 도메인(`nginx.test.local`)으로 접속하기 위해 실습용 클라이언트(SUSE Linux)의 `/etc/hosts` 파일에 워커 노드의 IP와 도메인을 매핑해 줍니다.

- **Cluster > Nodes** 메뉴로 이동해, 노드(`sr-downstream-XX`) IP를 확인합니다.

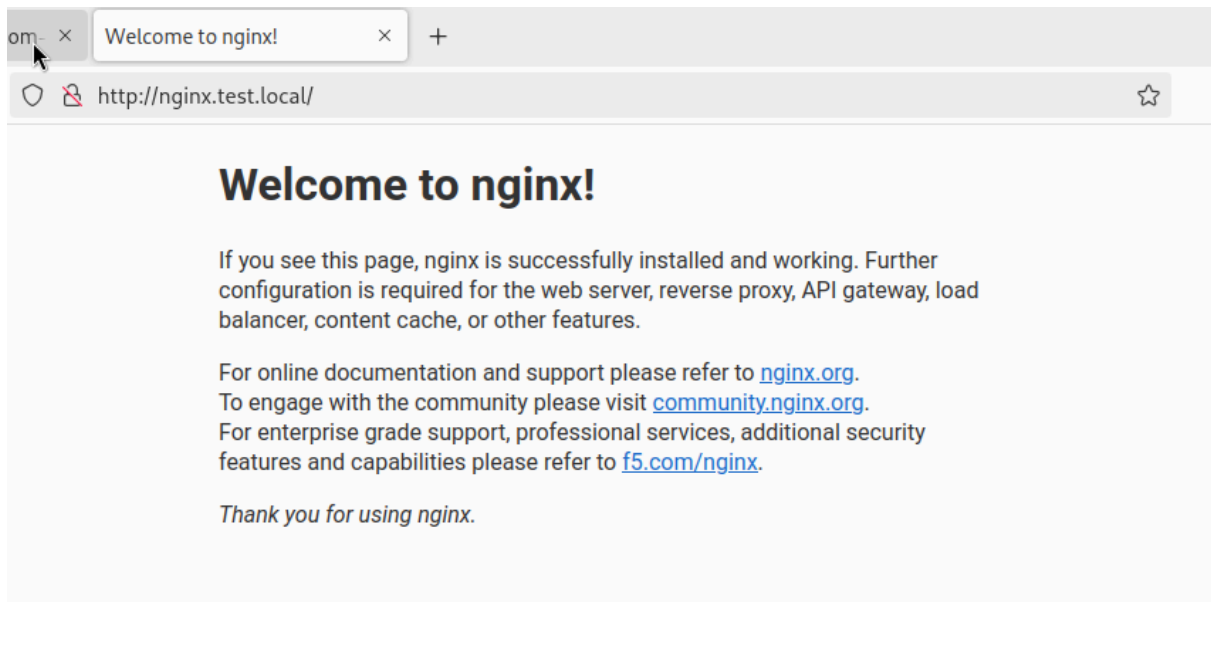


- 터미널에서 `/etc/hosts` 파일을 열고 `vi /etc/test` 명령어를 실행해, `<IP>`
`nginx.test.local` 을 추가합니다.

```
localhost:~ # cat /etc/hosts
#
# hosts                This file describes a number of hostname-to-address
#                      mappings for the TCP/IP subsystem.  It is mostly
#                      used at boot time, when no name servers are running.
#                      On small systems, this file can be used instead of a
#                      "named" name server.
#
# Syntax:
#
# IP-Address  Full-Qualified-Hostname  Short-Hostname
#
127.0.0.1    localhost
#
# special IPv6 addresses
::1          localhost ipv6-localhost ipv6-loopback
fe00::0      ipv6-localnet
ff00::0      ipv6-mcastprefix
ff02::1      ipv6-allnodes
ff02::2      ipv6-allrouters
ff02::3      ipv6-allhosts
192.168.2.93 nginx.test.local
```

Step 4-6. 최종 접속 테스트

웹 브라우저를 열어 주소창에 `http://nginx.test.local/` 을 입력하고, **Welcome to nginx!** 화면이 정상적으로 출력되는지 확인합니다.



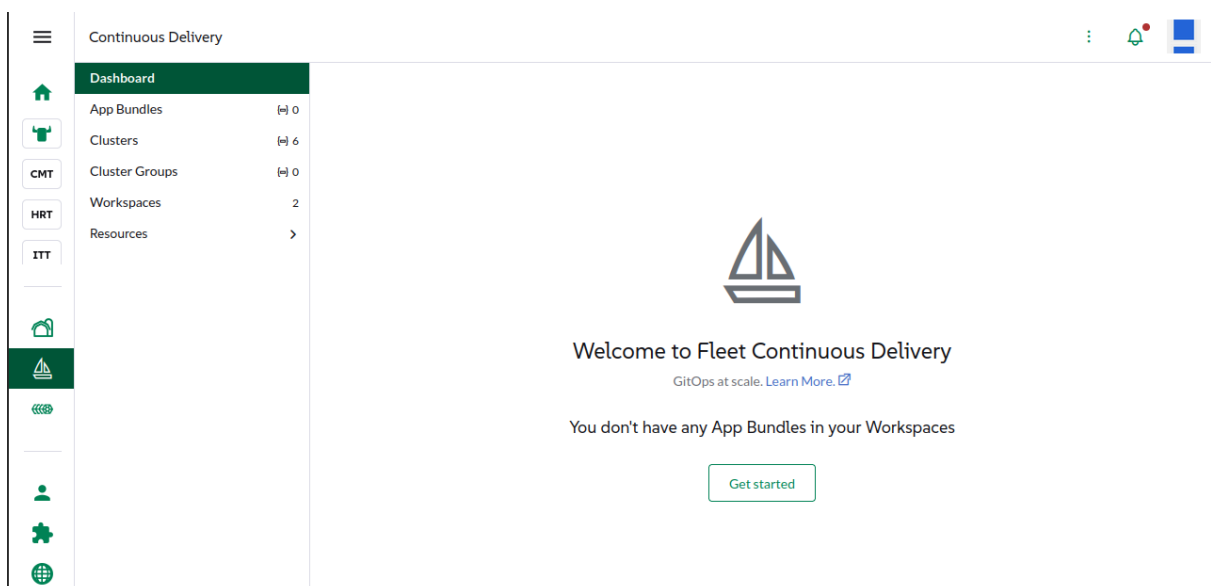
Step 5. Helm Chart를 통해 애플리케이션 배포하기

Rancher의 핵심 기능 중 하나인 Continuous Delivery(Fleet)를 활용하여, 외부 Helm Repository의 차트를 클러스터에 배포하는 과정을 실습합니다.

Step 5-1. Continuous Delivery 를 통한 애플리케이션 배포

햄버거 메뉴(좌측 상단)를 클릭하고 **Continuous Delivery**로 이동합니다.

초기 화면 중앙의 **Get started** (또는 우측 상단의 Create App Bundle) 버튼을 클릭합니다.



Git 저장소를 연동하는 방식과 Helm 차트를 직접 바라보는 방식 중, **Helm Ops**를 선택합니다.

App Bundle: Create

	Git Repos GitRepo describes a git repository that is watched by Fleet. The resource contains the necessary information to deploy the repo, or parts of it, to target clusters.
	Helm Ops HelmOp is a simplified way of creating bundles by directly pointing to a Helm repository or to an OCI registry, without needing to set up a git repository.

배포할 App Bundle의 이름과 라벨을 지정합니다.

- **Name:** `nginx-helm`
- **Labels:** `Key: app` / `Value: nginx-helm`

App Bundle: Create

Create: Step 1

Define metadata details

Metadata Chart Values Target details Advanced

Name *

nginx-helm

Description

Any text you want that better describes this resource

Labels

Key/value pairs that are attached to objects which specify identifying attributes.

Key

app

Value

nginx-helm

Remove

+ Add Label

Annotations

+ Add Annotation

Cancel

Edit as YAML

Next

Helm Chart 저장소와 Nginx 차트 정보를 입력합니다.

- **Name:** nginx-helm
- **Type:** Repository
- **Repository URL:** https://charts.bitnami.com/bitnami
- **Chart:** nginx
- **Version:** 23.0.0

App Bundle: Create

Create: Step 2

Define Helm Chart source details

Metadata **Chart** Values Target details Advanced

Release

Name
nginx-helm

Source

Type
Repository

Repository URL *
https://charts.bitnami.com/bitnami

Chart *
nginx

Version
23.0.0

Cancel

Edit as YAML

Previous

Next

차트 배포 시 적용할 환경 설정(`values.yaml`)을 정의합니다. NodePort와 Ingress를 함께 구성하도록 아래와 같이 작성합니다.

```
service:
  type: NodePort
ingress:
  enabled: true
  ingressClassName: nginx
  hostname: "nginx.helm.local"
  path: /
  pathType: Prefix
```

App Bundle: Create

Create: Step 3

Define Helm Values for the chart

Metadata Chart **Values** Target details Advanced

Configure Values used by Helm that help define the App.

Values

```
1 * service:
2   type: NodePort
3 * ingress:
4   enabled: true
5   ingressClassName: nginx
6   hostname: "nginx.helm.local"
7   path: /
8   pathType: Prefix
```

Values From

Add Variable

Cancel

Edit as YAML

Previous

Next

이 앱을 어느 클러스터, 어느 네임스페이스에 배포할지 지정합니다.

- **Deploy To:** **Manually selected clusters** 선택
- **Select by name:** 현재 실습 중인 클러스터(**custom-test**) 선택
- **Target Namespace:** **demo-app**

App Bundle: Create

Create: Step 4

Define target details

Metadata Chart Values **Target details** Advanced

Deploy To

- ☐ All Clusters in the workspace
- ☐ No clusters
- ☒ Manually selected clusters

Select by name

Clusters
custom-test x

Selected clusters
custom-test

Select by labels

+ Add cluster selector

Additional Options

Service Account Name
Optional: Use a service account in the target clusters

Target Namespace
demo-app

Cancel

Edit as YAML

Previous

Next

추가적인 인증이나 리소스 세팅을 하는 단계입니다. 이번 실습에서는 Accept any certificate (insecure)의 체크박스만 체크한 후 우측 하단의 **Create** 버튼을 클릭하여 생성을 완료합니다.

App Bundle: Create

Create: Step 5
Define advanced settings

Metadata

Chart

Values

Target details

Advanced

These settings are for advanced usage and optional. You can skip them and finish the creation process.

Authentication

Helm Authentication

None

☒ Accept any certificate (insecure)

Resources

Secrets

Config Maps

☐ Enable self-healing

☐ Always keep resources

Polling

☐ Enable Polling

Cancel

Edit as YAML

Previous

Create

App Bundles 목록 화면으로 전환되며, 방금 만든 **nginx-helm**의 상태가 **Active**로 변경되는 지 확인합니다. (Resources 숫자가 올라가며 배포가 진행됩니다.)

App Bundles

Create App Bundle

Pause

Download YAML

Delete

Menu

Filter

Filter

☐ State	Name	Type	Source	Target	Clusters Ready	Resources	Age	
☐ Active	nginx-helm	HelmOp	charts.bitnami.com/bitnami nginx: 23.0.0	Clusters	1/1	7	45 secs	

좌측 메뉴의 **Dashboard**로 이동하여, Workspace에 1개의 App Bundle이 정상적으로 동작 중인지 전체 현황을 파악합니다.

Continuous Delivery

Dashboard

- App Bundles 1
- Clusters 6
- Cluster Groups 0
- Workspaces 2
- Resources >

Continuous Delivery Dashboard

Workspace: **fleet-default**

1 App Bundle ✓ 1
6 Clusters ✓ 6

Filter

☒ Show Git Repos
☒ Show Helm Ops

Create App Bundle

> Active 1 / 1

Workspace: **fleet-local**

1 Cluster ✓ 1
1 Cluster Group ✓ 1

Step 5-2. 클러스터 내 애플리케이션 배포 확인

다시 햄버거 메뉴(좌측 상단)를 클릭하고 실습 클러스터(**custom-test**)로 돌아온 뒤, **Workloads > Deployments** 메뉴로 이동합니다.

custom-test

demo-appX

Cluster >

Workloads >

- CronJobs 0
- DaemonSets 0
- Deployments 2**
- Jobs 0
- StatefulSets 0
- Pods 3
- Apps >

Deployments ☆

Create

Redeploy Download YAML Delete

☐	State	Name	Namespace	Image	Ready	Up To Date	Available	Age	
☐	Active	nginx-demo	demo-app	nginx	2/2	2	2	4.5 hours	
☐	Active	nginx-helm	demo-app	registry-1.docker.io/bitnami/nginx	1/1	1	1	1.4 mins	

좌측 메뉴에서 **Service Discovery > Services** 로 이동하여 **Service Type** 을 **NodePort** 로 지정했던 부분이 잘 적용되었는지 확인합니다.

custom-test

demo-appX

Cluster >

Workloads >

Apps >

Service Discovery >

- HorizontalPodAutoscalers 0
- Ingresses 2
- Services 2**
- Storage >
- Policy >
- More Resources >

Services ☆

Create

Download YAML Delete

☐	State	Name	Namespace	Target	Selector	Type	Age	
☐	Active	nginx-demo	demo-app	31521/TCP	app=nginx-demo	Node Port	4.5 hours	
☐	Active	nginx-helm	demo-app	30853/TCP 32559/TCP	app.kubernetes.io/instance=nginx-helm app.kubernetes.io/name=nginx	Node Port	1.9 mins	

생성한 Service(**nginx-helm**)을 클릭하면 해당 Service의 상세 페이지로 이동합니다.

Service: nginx-helm Active Show Configuration

Namespace: demo-app Labels: 5 Annotations: 2

Age: 1.1 mins Type: NodePort Cluster IP: 10.43.39.27 Session Affinity: None

Labels: app.kubernetes.io/instance: nginx-helm, app.kubernetes.io/managed-by: Helm, app.kubernetes.io/name: nginx

Annotations: meta.helm.sh/release-name: nginx-helm, meta.helm.sh/release-namespace: demo-app

Pods (1) Ports (2) Selectors (2) Conditions (0) Recent Events (0) Related Resources (2)

State	Name	Namespace	Image	Ready	Restarts	IP	Node	Age
Running	nginx-helm-5748994bf9-8hrzs	demo-app	registry-1.docker.io/bitnam/nginx	1/1	0	10.42.231.225	sr-downstream-99	14 hours

Ports 탭에서 http(80) 과 연결된 NodePort를 확인한 이후 웹페이지에서 <IP>: <NodePort> 를 작성하여 Nginx NodePort 테스트를 합니다.

Service: nginx-helm Active Show Configuration

Namespace: demo-app Labels: 5 Annotations: 2

Age: 1.1 mins Type: NodePort Cluster IP: 10.43.39.27 Session Affinity: None

Labels: app.kubernetes.io/instance: nginx-helm, app.kubernetes.io/managed-by: Helm, app.kubernetes.io/name: nginx

Annotations: meta.helm.sh/release-name: nginx-helm, meta.helm.sh/release-namespace: demo-app

Pods (1) **Ports (2)** Selectors (2) Conditions (0) Recent Events (0) Related Resources (2)

Name	Port	Protocol	Target	Node Port	Public Ports
https	443	TCP	https	32364	
http	80	TCP	http	32730	

stom-t × Welcome to nginx! × +

http://192.168.2.93:30853/ ☆

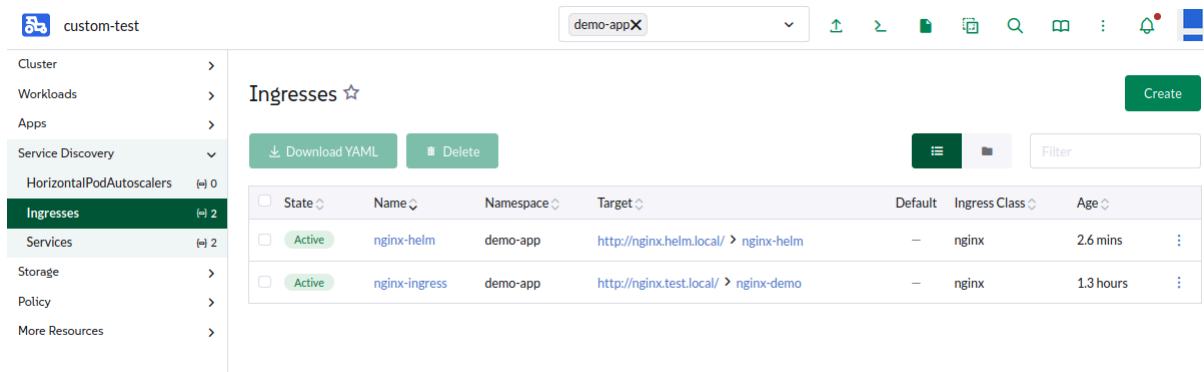
Welcome to nginx!

If you see this page, nginx is successfully installed and working. Further configuration is required for the web server, reverse proxy, API gateway, load balancer, content cache, or other features.

For online documentation and support please refer to nginx.org.
To engage with the community please visit community.nginx.org.
For enterprise grade support, professional services, additional security features and capabilities please refer to f5.com/nginx.

Thank you for using nginx.

좌측 메뉴에서 **Service Discovery > Ingresses**로 이동하여 `nginx-helm` Ingress가 생성되어 있고, Target 도메인이 `http://nginx.helm.local/`, Ingress Class가 `nginx`로 정확하게 들어간 것을 확인합니다.



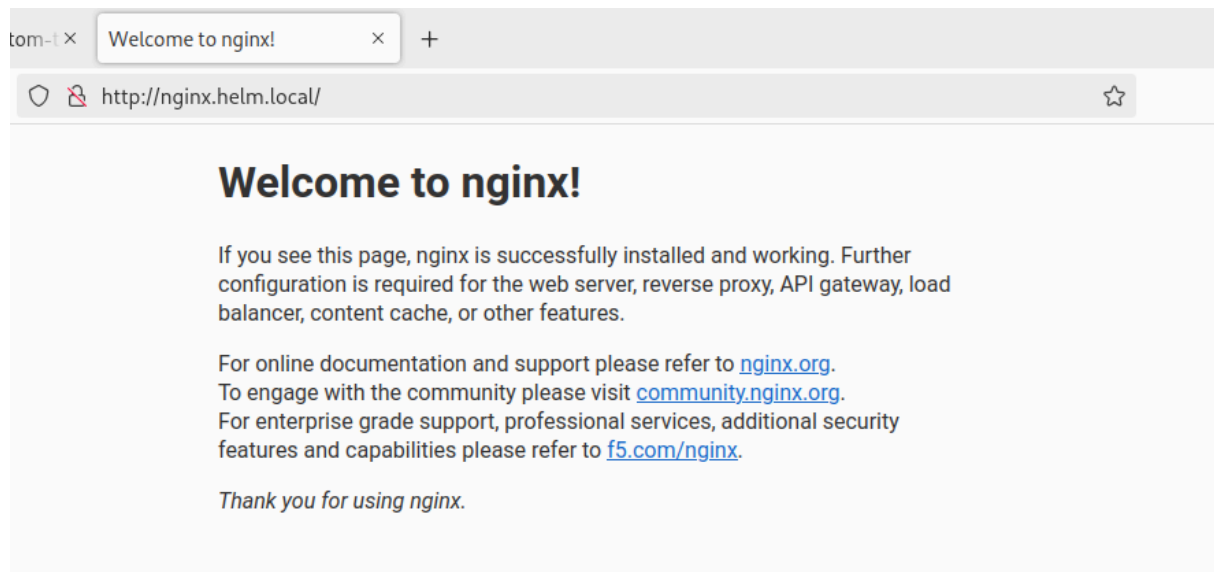
새롭게 배포한 Ingress 도메인(`nginx.helm.local`)으로 접속하기 위해 `/etc/hosts` 파일을 수정합니다.

```

+
localhost:~ # cat /etc/hosts
#
# hosts          This file describes a number of hostname-to-address
#                mappings for the TCP/IP subsystem.  It is mostly
#                used at boot time, when no name servers are running.
#                On small systems, this file can be used instead of a
#                "named" name server.
#
# Syntax:
#
# IP-Address     Full-Qualified-Hostname  Short-Hostname
#
127.0.0.1        localhost
#
# special IPv6 addresses
::1              localhost ipv6-localhost ipv6-loopback
fe00::0          ipv6-localnet
ff00::0          ipv6-mcastprefix
ff02::1          ipv6-allnodes
ff02::2          ipv6-allrouters
ff02::3          ipv6-allhosts
192.168.2.93     nginx.helm.local
localhost:~ #

```

웹 브라우저 주소창에 `http://nginx.helm.local/` 을 입력하여 NodePort 접속 때와 동일하게 **Welcome to nginx!** 페이지가 정상적으로 출력되는지 확인합니다.



4. 성공 기준

#	기준	확인
1	<code>hands-on-project</code> Project 생성 완료	☐
2	<code>demo-app</code> Namespace 생성 완료	☐
5	<code>nginx-demo</code> Deployment, Pod, Service 정상 확인 및 브라우저 접속 성공	☐
4	nginx Helm Chart 설치 완료 (<code>nginx-helm</code> Deployed 상태)	☐
5	Rancher UI에서 Workload 상태 확인 가능	☐

5. 실습 결과 요약

- **Project/Namespace 관리**: Rancher의 멀티테넌시 구조인 Project와 Namespace 를 생성하여 워크로드를 논리적으로 분리했습니다.
- **애플리케이션 배포** : Rancher Dashboard의 Continuous Delivery 메뉴를 통해 GUI 만으로 컨테이너 애플리케이션을 배포하는 과정을 수행했습니다.
- **애플리케이션 외부 노출** : Service 및 Ingress를 통해 애플리케이션을 외부로 노출하는 방법을 익혔습니다.

👉 **다음 Session 안내:** Session 5에서는 RBAC(역할 기반 접근 제어)를 설정하여 사용자 및 권한을 관리합니다. Rancher Dashboard 접속 상태와 Downstream Cluster(`custom-test`)의 현재 상태를 유지해 주세요.

👉 **다음 세션으로 넘어가기**

👉 **메인 페이지로 돌아가기**